

Раздел 1. Основы объектно-ориентированного проектирования (ООП)

1. Инкапсуляция, наследование, полиморфизм: Дайте определения. Чем полиморфизм подтипов (через наследования) отличается от параметрического (дженерики, шаблоны)?
2. Абстракция и сокрытие информации: В чем разница? Приведите пример «дырявой абстракции».
3. Ассоциация, агрегация, композиция: Сравните виды отношений. Как семантика времени жизни объекта влияет на выбор между агрегацией и композицией?
4. Наследование vs Делегирование: Сравните принципы. В каком случае делегирование предпочтительнее наследования (принцип предпочитайте композицию наследованию)?
5. Принцип подстановки Барбары Лисков (LSP) – связующее звено между ООП и SOLID.
6. Через SOLID поясните какие принципы ООП реализуются и контролируются.

Раздел 2. Язык UML (диаграммы, нотация, назначение)

6. Структурные и поведенческие диаграммы: Перечислите по 2 примера на каждую категорию. Для чего нужна Диаграмма компонентов?
7. Диаграмма классов: Подробно: что означают +, -, #, ~, роль конца ассоциации (multiplicity), навигация (стрелка), квалификатор.
8. Диаграмма последовательности (Sequence): Фреймы (alt, opt, loop, ref), синхронные и асинхронные вызовы, «кирпичи жизни» (activation bars).
9. Диаграмма состояний (State Machine): Входные/выходные действия (entry/exit), внутренние переходы, суперсостояния (ортогональные).
10. Диаграмма деятельности (Activity): Чем отличается от блок-схемы? Объясните семантику узлов принятия решений, слияния, форков и JOIN. Пинны (pins).

11. Диаграмма вариантов использования (Use Case): Что такое extend и include? Приведите пример, когда использовать extend с условием (extension point).
12. Диаграмма развертывания (Deployment): Артефакты, узлы (device vs execution environment), стереотипы типа «database», «clientServer».

Раздел 3. Парадигма SOLID (5 принципов)

13. Single Responsibility (SRP): Почему «наличие одной причины для изменения» — более точная формулировка, чем «один метод»?
14. Open-Closed (OCP): Объясните механизм «открытости для расширения» через абстракцию. Пример нарушения (класс с цепочкой if-else instanceof).
15. Liskov Substitution (LSP): Объясните на примере класса Rectangle и Square, почему наследование в реальном мире не равно наследованию в ООП (проблема контравариантности предусловий/постусловий).
16. Interface Segregation (ISP): Чем опасны «толстые интерфейсы»? Сравните с проблемой God Object.
17. Dependency Inversion (DIP): В чем разница между внедрением зависимости и инверсией зависимости? Почему высокоуровневые модули не должны зависеть от низкоуровневых?

Раздел 4. GRASP (паттерны распределения обязанностей)

18. Information Expert: Как решить, какой класс должен создать новый объект? Сравните с Creator (когда В создает А?).
19. Low Coupling (низкое зацепление) vs High Cohesion (высокая связность): Приведите пример конфликта этих принципов и как его разрешить.
20. Controller: Что должен делать контроллер (UI-слой)? Чем фасадный контроллер отличается от контроллера для одного варианта использования?

21. Polymorphism vs Protected Variations: Как эти паттерны связаны? Объясните на примере поддержки разных форматов платежей (Visa, PayPal, Crypto).

22. Pure Fabrication: В чем отличие от Indirection? Когда создание «искусственного» класса оправдано, а когда это преждевременное усложнение?

Раздел 5. GoF (Gang of Four): Порождающие паттерны

23. Factory Method vs Abstract Factory: Ситуации использования. Что проще добавить: новый продукт или новое семейство продуктов?

24. Singleton: Проблемы в многопоточной среде. Двойная проверка блокировки (Double-Checked Locking) — работает ли в Java/C#?

25. Builder: Чем отличается от паттерна Step Builder (fluent interface)? Когда нужен директор (Director)?

26. Prototype: Проблема глубокого и поверхностного клонирования. Как быть с циклическими ссылками?

Раздел 6. GoF: Структурные паттерны

27. Adapter vs Bridge vs Прoxy: Ключевая разница:

28. Decorator vs Composite: Оба работают с деревьями/вложенностью. Чем отличается намерение (intent) и как это влияет на реализацию (super vs обход детей)?

29. Facade vs Mediator: Фасад упрощает интерфейс (знает всех), а медиатор — централизует взаимодействие (знает всех, и они знают только его).

Подробно.

30. Flyweight: Условия применимости: когда объекты имеют внутреннее и внешнее состояние? Пример из графики (символы, текстуры).

Раздел 7. GoF: Поведенческие паттерны

- 31. Strategy vs State: UML-диаграмма у них одинаковая. Как различить их семантику по поведению контекста (меняется ли стратегия извне или сама автоматом)?
- 32. Observer vs Mediator vs Pub-Sub: Чем EventBus отличается от классического Observer? Проблема утечек памяти (долгоживущий наблюдатель).
- 33. Command vs Memento: Как связка этих паттернов реализует Undo/Redo? Как Memento нарушает инкапсуляцию и как это лечат?
- 34. Chain of Responsibility: Напоминает if-else в цикле. В чем выгода? Пример из middleware в веб-фреймворках.
- 35. Template Method vs Strategy: Шаблонный метод фиксирует скелет алгоритма (наследование), Стратегия — переопределяет всю логику (композиция). Когда какой выбрать?

Раздел 8. Синтез и архитектурные вопросы

- 36. MVC как композиция паттернов: Какие GoF паттерны лежат в основе Model-View-Controller? Обоснуйте (View как Observer, Controller как Strategy и т.д.).
- 37. Антипаттерны: Что такое God Object, Lava Flow (латентный код), Poltergeist (временный класс-однодневка). Как их выявить через метрики (например, LCOM — Lack of Cohesion of Methods)?
- 38. Закон Деметры (LoD): Запрет на цепочки a.getB().getC().do(). Как его обойти через Tell, Don't Ask?
- 39. Сравнение GRASP Controller и GoF Facade: Могут ли они сосуществовать в одном проекте?
- 40. От варианта использования к коду: Опишите путь: «Use Case → Диаграмма последовательности → Диаграмма классов → Код, использующий 2 порождающих паттерна».